OPEN-SOURCE LLMs · DAY 2

The model comes to you

By minute 20, a language model is running on your laptop — no internet, no API key, no bill.

```
ollama --version
```

Terminal check — do it now. Version number = you're set.

A model is a file

Open weights vs closed — where the intelligence actually lives.

Open weights

The model's file is published. Download it, run it, forever.

```
qwen3-…-Q4_K_M.gguf ≈ a few GB = the whole model
```

Closed

The file stays on their servers (ChatGPT, Claude, Gemini). You rent access through a window.

Go look: huggingface.co → search "Qwen3 GGUF" → Files tab. A whole model is smaller than one season of a show.

Pull now — talk while it downloads

Ollama runs model files like a music player; the .gguf is the song.

```
ollama pull qwen3:0.6b      # ~520 MB — any laptop
ollama pull mistral:7b     # 16 GB RAM? second tab, for later
```

- pull downloads from a registry — exactly like docker pull / npm install.
- qwen3:0.6b reads as: family Qwen3, size 0.6 billion weights.
- Keep the progress bar visible — that bar is ~600 million learned numbers arriving on your machine.
- Blocked corporate laptop? The hosted provider (Lab 1) covers every lab — same API shape.

What is a "weight"?

One learned number. A 7B model is a web of seven billion of them — and nobody writes them.

Training finds the numbers.
Start random → hide the last word of a sentence → nudge the numbers that would make the right word likelier → repeat, trillions of times.

Analogy: seven billion tuning pegs, all random (noise). A robot tuner nudges pegs until it's music. The file you're downloading IS the final peg positions — not the music, not the training data.

It explains three things

1 · File size = numbers × bytes-each.

2 · "If you don't tell it, it doesn't know" — facts are smeared across the numbers, not stored in rows.

3 · Quantization = rounding each peg slightly; with billions voting, the music barely changes.

FIRST LOCAL TOKEN — minute 20, promise kept

```
ollama run qwen3:0.6b     # ask it anything · exit with /bye
```

Ninety seconds of play:

- Throw a Day-1 six-ingredient prompt at it — does it obey?
- Ask something after its training cutoff — watch it hallucinate confidently.
- The dare: turn off your wifi and ask again. Still works. No internet, no key, no meter.

That is a language model running on your own hardware. Everything else today just makes it more useful.

Who makes these? (and why the table has a shelf life)

Written in May. By July almost everything here had changed.

- June 2026 — six Chinese frontier releases in two weeks; GLM became the #1 open model (MIT).
- Meta — who started open weights — shelved its big model and delayed Llama 5 to 2027, going closed.
- July — Kimi K3 shipped at 2.8 trillion parameters, the largest open model ever.
- Chinese labs now hold five of the top-ten open slots.

The lesson: any model table is stale in ~8 weeks. So we don't teach the leaderboard — we teach six questions that never expire.

Mixture of Experts — read the numbers

"753B / 40B" = 753 billion weights total, but only 40 billion wake up per token.

Speed & cost
follow the SMALL number (active params).

Download & RAM
follow the BIG number (total params). Marketing quotes the big one.

The hospital
The model is a building with hundreds of specialists. Your question sees a handful — not the whole staff. Huge hospital, small bill.

Almost every 2026 flagship is MoE — e.g. Kimi K3: 2.8T total, ~16 of 896 experts active (prices like a ~50B model).

The map — recognise, don't memorise

One column outlives every benchmark: LICENSE.

GLM	#1 open model · MIT
Kimi	the 2.8T giant
DeepSeek	reasoning & math · MIT
Qwen	multilingual all-rounder
Llama	the old king — now closed frontier
Gemma	Google's laptop family
MiniMax	the agentic newcomer

✓ *MIT / Apache = do-anything.* ⚠ *others = read before commercial use.* *Refresh names from ollama.com/search — the table ages fast.*

Six questions to pick any model

The forever-skill. Walk the drill out loud.

1. **Task** — what job is it doing?
2. **License** — can you legally use it?
3. **Hardware** — does it fit your machine?
4. **Latency** — interactive or batch?
5. **Quality bar** — must it beat the frontier, or is good-enough fine?
6. **Hosting** — laptop, your server, or hosted?

Drill: HR chatbot · sensitive data · 200 users · small budget → self-hosted, MIT/Apache, 7–13B, good-enough.

LAB 1 · Cross-model bake-off — 20 min

Your hosted provider · pick 3 models from 3 families · same prompt · rank for THIS task.

```
Explain how Mixture of Experts (MoE) works
in exactly 3 bullets, plain English, no jargon,
each bullet under 25 words.
```

Suggested (Groq)
`llama-3.3-70b-versatile`
`deepseek-r1-distill-llama-70b`
`qwen/qwen3-32b`

- Write your predicted winner first (a bet) — upsets are common.
- Note latency, output quality, output length. Rank them.
- Three rankings across the room will disagree — that's the point: the bake-off beats the leaderboard for YOUR task.

Match model to machine

Quantization = 4-bit weights ("Q4"): ~4× smaller, barely dumber. JPEG for neural nets.

8 GB RAM, no GPU	up to ~3B · qwen3:0.6b (you have it)
16 GB RAM	the sweet spot · mistral:7b / llama3.1:8b
Discrete GPU 8GB+	~13B in reach
Apple Silicon	unified memory punches above its weight

Rule of thumb: 7B at Q4 ≈ 4–5 GB of RAM/VRAM. Small laptop? Zero shame — every lab works on 0.6b.

LAB 2 · Python + Ollama ladder — 25 min

```
import requests

def ask(prompt, system=None, model="qwen3:0.6b"):
    msgs = ([{"role":"system","content":system}]
            if system else []) \
           + [{"role":"user","content":prompt}]
    r = requests.post(
        "http://localhost:11434/v1/chat/completions",
        json={"model":model,"messages":msgs})
    return r.json()["choices"][0]["message"]["content"]
```

The rungs

1 · first local reply
2 · add a system message
3 · force JSON out
4 · same code → hosted
5 · swap model="mistral:7b"

`localhost:11434` — local, but the SHAPE is the OpenAI dialect every cloud speaks. Learn it once, drive anything. Climb as far as you get; no pressure to finish.

Debrief — screens down, voices up

- Latency — who was faster, local or hosted? First token vs whole answer?
- Quality — did 0.6B obey the six ingredients? Where did it crack?
- JSON — who got valid JSON locally, and what did it take?
- Swap — who ran two models with the same code?

The honest trade — write this down. Local is private, free, offline, YOURS. Hosted is stronger, faster, zero-ops. Professionals use both — the six questions tell you when.

Where this goes — agents (names only)

Models answer; agents act — call tools, loop, work multi-step.

MCP	the wiring standard between any model and any tool — you'll build a server later this week; your local model becomes its engine
LangGraph	workflows as graphs — the production choice
CrewAI	agents as a team with roles — the easy start
Pydantic AI	type-safe agents — today's JSON skill, weaponised

Recognise the names. MCP day does the rest, hands-on.

Cheat sheet — pin it near your screen

The six questions (forever-skill)

- Task
- License
- Hardware
- Latency
- Quality bar
- Hosting

Local quick-reference

```
ollama pull <model>
ollama run <model>
ollama serve
ollama list
```

Rule of thumb: 7B at Q4 ≈ 4–5 GB.

Your laptop can think now.

Nobody can meter it, rate-limit it, or take it away. The model table goes stale by autumn; the capability and the six questions don't.

Next — MCP day: today's local model becomes an agent's engine. Questions?